```
module fir_filter(
input clk, // Clock signal
input reset, // Reset signal (asynchronous)
input [15:0] x, // Input data (16 bits)
output reg [31:0] y // Filter output (32 bits, to hold the accumulated result)
);
// Parameter: Number of taps in the FIR filter
parameter N = 4; // Filter with 4 taps
// Coefficients for the FIR filter (example values, can be adjusted)
reg signed [15:0] h[0:N-1];
// Shift register to store the input samples (delay line)
reg signed [15:0] x_reg [0:N-1];
reg [2:0] count;
// Always block to implement the FIR filter
always @(posedge clk or posedge reset) begin
if (reset) begin
// Reset filter output and shift register
y <= 32'd0;
h[0] <= 16'sh4;
h[1] <= 16'sh3;
h[2] <= 16'sh2;
h[3] <= 16'sh1;
x_reg[0] <= 16'h0;
x_reg[1] <= 16'h0;
x_reg[2] <= 16'h0;
x_reg[3] <= 16'h0;
end
else begin
// Shift input data into the delay line
x_reg[3] <= x_reg[2]; // Shift previous input
x_reg[2] <= x_reg[1]; // Shift previous input
```

```
x_reg[1] <= x_reg[0]; // Shift previous input
x_reg[0] <= x; // Store the new input at the first position
// Perform the FIR filter calculation (multiply and accumulate)

if(count < 3'b100)
y <= y + (h[count] * x_reg[count]);
end
end
always @(posedge clk or posedge reset)
if(reset)
count <= 3'b0;
else
count <= count + 3'b1;
endmodule
```